

Using PostgreSQL with Node.js (via the native pg driver) is **conceptually very similar** to using MySQL (via mysql2) — especially when you're using:

- **Raw SQL queries**
- **Async/Await**
- **No ORM** like Sequelize or Prisma

Let me break it down clearly for comparison:

Similarities between MySQL (mysql2) & PostgreSQL (pg) in Node.js

Feature	MySQL (mysql2/promise)	PostgreSQL (pg)
Async/Await support	✓ Yes (via .promise())	✓ Yes (native async with pg)
Raw query execution	await db.execute('SQL', [...])	await db.query('SQL', [...])
Result structure	const [rows] = await db.execute(...)	const result = await db.query(...); result.rows
No automatic table creation	✗ Needs manual CREATE TABLE	✗ Needs manual CREATE TABLE
SQL injection safe (param use)	✓ Uses ? placeholders	✓ Uses \$1, \$2 placeholders
Setup with .sql file	✓ mysql -u root -p < schema.sql	✓ psql -U user -f schema.sql
Pool support	✓ mysql2.createPool().promise()	✓ pg.Pool()

Key Differences

Thing	MySQL	PostgreSQL
Param placeholders	?, e.g. WHERE email = ?	\$1, \$2, e.g. WHERE email = \$1
Auto increment	AUTO_INCREMENT	SERIAL or GENERATED ALWAYS AS IDENTITY
Boolean type	TINYINT(1)	BOOLEAN
Returning inserted row	Not returned unless fetched separately	Use RETURNING * to get it instantly

Yes, you can absolutely use destructuring when you retrieve data from the result of a PostgreSQL query!

When using the pg library in Node.js, the query() method returns an object that contains the rows property, which is an array of results. So, you can destructure it as follows:

js

CopyEdit

```
const { rows } = await db.query('SELECT * FROM users WHERE email = $1', [email]);
```

This is equivalent to:

js

CopyEdit

```
const result = await db.query('SELECT * FROM users WHERE email = $1', [email]);
```

```
const rows = result.rows;
```

In both cases, you're retrieving the data inside the rows array from the query result.

Example:

js

CopyEdit

```
const { rows } = await db.query('SELECT * FROM users WHERE email = $1', [email]);
```

```
// Now `rows` contains all matching users
```

```
if (rows.length > 0) {
```

```
  console.log('User found:', rows[0]);
```

```
} else {
```

```
  console.log('No user found.');
```

```
}
```

Recap:

- **Destructuring** allows you to directly extract properties (like rows) from objects returned by queries.
- The result of db.query() will be an object that includes the rows (data) and additional metadata (like rowCount, etc.).

In PostgreSQL, the `\c` command is a **meta-command** used in the **psql** command-line interface to **connect to a different database**. It is specific to the **psql shell** and is not part of the SQL language itself.

Explanation:

- `\c` is shorthand for "connect" in psql, and it allows you to change the current active database without leaving the psql shell.
- It is used when you're already inside the psql command-line tool and want to switch to another database, such as:

bash

CopyEdit

```
\c node_app;
```

This will connect to the `node_app` database in PostgreSQL.

Why it's different from MySQL:

- **MySQL** does not have a direct equivalent of `\c` within the command line. In MySQL, switching databases is done with the `USE` command:

sql

CopyEdit

```
USE node_app;
```

This is an **SQL command** and is not a part of the MySQL client's internal meta-commands.

Summary:

- **PostgreSQL (psql):** `\c` is a meta-command to change the current database inside the psql shell.
- **MySQL:** `USE` is an SQL command to switch databases in the MySQL shell.